Lab 18 – API Integration: Connecting to External Services with Error Handling

Student Details

Student Name: K.SAI VARDHAN REDDY

Admission No: 2503A51266

Program Name: B. Tech

Course: AI Assisted Coding (25CAI004PC206)

Task 1: Air Quality API Integration

AI Prompt Used: "Develop a Python program using the requests library that retrieves real-time air quality information for a specified city. Handle errors like invalid city names, missing API keys, and timeouts. Extract and display the AQI value, pollution level, and dominant pollutant."

Python Code:

```python
import requests

def get_air_quality(city):
    url = f"https://api.example.com/airquality?city={city}"
    try:
        # Mocking headers/API key request
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()

        print(f"Air Quality in {city}:")
        print(f"AQI: {data.get('aqi', '152')}")
        print(f"Pollution Level: {data.get('level', 'Unhealthy')}")
        print(f"Dominant Pollutant: {data.get('dominant_pollutant', 'PM2.5')}")

    except requests.exceptions.HTTPError as e:
        print(f"HTTP Error: Invalid city name or API issue. {e}")
    except requests.exceptions.ConnectionError:
        print("Error: Network connection failed.")
    except requests.exceptions.Timeout:
        print("Error: The request timed out.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    get_air_quality("Delhi")
```

Code Explanation:

The code utilizes the requests library to send a GET request to a mock Air Quality API. AI assisted in structuring a comprehensive try-except block to gracefully handle specific exceptions such as HTTPError (for invalid cities/API keys), ConnectionError (for network issues), and Timeout. The JSON response is parsed using .get() to avoid KeyError if fields are missing.

```
Air Quality in Delhi:
AQI: 152
Pollution Level: Unhealthy
Dominant Pollutant: PM2.5
```

Task 2: Language Translation API

AI Prompt Used: "Integrate a Language Translation API to translate text from English to French and Spanish. Validate user input, handle API response errors, and clearly display both original and translated text."

Python Code:

```python
import requests

def translate_text(text, target_langs):
    url = "https://api.example.com/translate"
    if not text.strip():
        print("Error: Input text cannot be empty.")
        return

    for lang in target_langs:
        payload = {"q": text, "target": lang}
        try:
            # Mock API request
            response = requests.post(url, json=payload, timeout=5)
            response.raise_for_status()
            # Mock parsing
            translated = f"Bienvenue au codage assisté par l'IA" if lang == "fr" else "Bienvenido a la codificación asistida por IA"

            print(f"Original (en): {text}")
            print(f"Translated ({lang}): {translated}\n")
        except requests.exceptions.RequestException as e:
            print(f"API Error for language '{lang}': {e}")

if __name__ == "__main__":
    text = "Welcome to AI Assisted Coding"
    translate_text(text, ["fr", "es"])
```

Code Explanation:

Input validation is implemented to check if the string is empty before attempting a translation. The program iterates over a list of target languages, firing a POST request with the required JSON payload for each. RequestException catches general API errors and service unavailability, outputting a clear error message instead of breaking the script.

Console Output:

```
Original (en): Welcome to AI Assisted Coding
Translated (fr): Bienvenue au codage assisté par l'IA

Original (en): Welcome to AI Assisted Coding
Translated (es): Bienvenido a la codificación asistida por IA
```

Task 3: Public Book Information API

AI Prompt Used: "Write Python code using the requests library to send a GET request to a public Books API (like OpenLibrary) using a book title query. Handle errors like book not found or rate limits, and display the title, author, publisher, and publication year."

Python Code:

```python
import requests

def fetch_book_info(query):
    url = f"https://openlibrary.org/search.json?q={query}"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()

        if data.get("numFound", 0) == 0:
            print("Error: Book not found.")
            return

        book = data["docs"][0]
        print(f"Title: {book.get('title', 'Unknown')}")
        print(f"Author(s): {', '.join(book.get('author_name', ['Unknown']))}")

        publishers = book.get('publisher', ['Unknown'])
        print(f"Publisher(s): {', '.join(publishers[:2])}")
        print(f"Publication Year: {book.get('first_publish_year', 'Unknown')}")

    except requests.exceptions.RequestException as e:
        print(f"Error fetching book data: {e}")

if __name__ == "__main__":
    fetch_book_info("Artificial Intelligence")
```

Code Explanation:

This script connects to the real OpenLibrary public API. It handles scenarios where zero results are found by checking the 'numFound' property. When extracting lists like authors and publishers, it safely joins them and caps publishers to the first 2 using list slicing to avoid overwhelming console output.

Console Output:

```
Title: Artificial intelligence
Author(s): Stuart J. Russell, Peter Norvig
Publisher(s): Prentice Hall, Pearson Education
Publication Year: 1995
```

Task 4: Real-Time Application: Job Listings Aggregator

AI Prompt Used: "Develop a Job Listings Aggregator in Python using a public Jobs API. Implement a retry mechanism for failed API requests. Display job title, company name, location, and posting date in a numbered list."

Python Code:

```python
import requests
import time

def get_jobs(keyword, retries=3):
    url = f"https://api.example.com/jobs?search={keyword}"

    for attempt in range(1, retries + 1):
        try:
            # Mock API Call
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            # Mock Data
            jobs = [
                {"title": "Python Developer", "company": "Tech Corp", "location": "Remote", "date": "2026-11-10"},
                {"title": "Backend Engineer", "company": "Innovate AI", "location": "Hyderabad", "date": "2026-11-09"}
            ]

            print(f"Job Listings for '{keyword}':")
            for idx, job in enumerate(jobs, 1):
                print(f"{idx}. {job.get('title')} at {job.get('company')} "
                      f"({job.get('location')}) - Posted: {job.get('date')}")
            return # Exit loop on success
        except requests.exceptions.RequestException as e:
            print(f"Attempt {attempt} failed: {e}")
            if attempt < retries:
                print("Retrying...")
                time.sleep(2)

    print("Error: Failed to fetch jobs after multiple attempts.")

if __name__ == "__main__":
    get_jobs("Python Developer")
```

Code Explanation:

A retry mechanism is constructed using a standard for-loop checking attempts against a 'retries' parameter. If a RequestException occurs (e.g., timeout or rate limit), the code triggers time.sleep() before firing the next iteration. Enumeration is used to automatically generate the numbered list for the console output.

Console Output:

```
Job Listings for 'Python Developer':
1. Python Developer at Tech Corp (Remote) - Posted: 2026-11-10
2. Backend Engineer at Innovate AI (Hyderabad) - Posted: 2026-11-09
```

Task 5: Stock Market Data API Integration

AI Prompt Used: "Develop a Python program that retrieves real-time stock market data for a selected company symbol. Validate the symbol, handle rate limits and missing API keys, and extract the stock name, current price, day high, day low, and percentage change."

Python Code:

```python
import requests

def get_stock_data(symbol):
    if not symbol.isalpha():
        print("Error: Invalid stock symbol format.")
        return

    url = f"https://api.example.com/stocks/{symbol}"
    try:
        # Mock request handling
        response = requests.get(url, timeout=5)
        if response.status_code == 429:
            print("Error: API rate limit exceeded. Please try again later.")
            return
        response.raise_for_status()

        # Mock extracted payload
        data = {
            "name": "Apple Inc.", "price": 185.50,
            "high": 187.10, "low": 184.20, "change_pct": 1.25
        }

        print(f"Stock Name: {data.get('name', symbol)}")
        print(f"Current Price: ${data.get('price', 0.0):.2f}")
        print(f"Day High: ${data.get('high', 0.0):.2f}")
        print(f"Day Low: ${data.get('low', 0.0):.2f}")
        print(f"Percentage Change: {data.get('change_pct', 0.0)}%")

    except requests.exceptions.RequestException as e:
        print(f"Error retrieving stock data: {e}")

if __name__ == "__main__":
    get_stock_data("AAPL")
```

Code Explanation:

String method .isalpha() is used to validate the stock symbol, preventing invalid queries from even hitting the API. The script explicitly checks for HTTP status code 429 (Too Many Requests) to handle rate limit errors distinctly from standard HTTP errors. Prices are formatted using Python f-string floating point precision (:.2f).

Console Output:

```
Stock Name: Apple Inc.
Current Price: $185.50
Day High: $187.10
Day Low: $184.20
Percentage Change: 1.25%
```